MARS: **Streaming How You Move with Motion-Aware ABR for Real-Time VR**

Anonymous Author(s)

Abstract

In this paper, we present MARS, a practical rate adaptation approach for real-time VR streaming. Delivering VR content over wireless networks costs much more bandwidth than traditional video streaming. Prior work primarily reduces the cost by exploiting users' *spatial viewing patterns* by prioritizing the visible region or streaming only the current viewport.

MARS reduces the bandwidth consumption from a different perspective by leveraging users' *temporal viewing patterns*, inspired by the observation that users' *dynamic visual acuity decreases during fast head motion*. To this end, MARS dynamically adjusts the bitrate based on viewers' head motions. However, realizing this is non-trivial and has two key challenges: (1) the head motion dynamics and bitrate adaptation should be precisely synchronized; (2) the bitrate adaptation should strike a careful balance between the bandwidth reduction and visual quality degradation. To address the challenges, MARS first proposes a kinematics-based synchronization mechanism to capture the viewers' motion phases for timely bitrate adaptation. It further employs a perceptually-guided bitrate adaptation algorithm to wisely reduce the bitrate by exploiting viewers' visual acuity. MARS provides a plug-and-play solution that can be readily integrated into existing VR streaming platforms. We prototype MARS and evaluate it through both real-world user studies and benchmarking. Results show that MARS delivers user experience comparable to a fixed-bitrate baseline while reducing bandwidth consumption by 10%-19%.

CCS Concepts

• **Human-centered computing** → **Virtual reality**; • **Information systems** → **Multimedia streaming**.

Keywords

Virtual reality, bandwidth reduction, motion-to-photon latency, quality of experience, rate adaptation

1 Introduction

Real-time wireless VR streaming is rapidly emerging as a key application for next-generation networks, enabling fully immersive, untethered experiences [4, 8, 32, 33]. However, delivering high-quality VR over WLAN remains challenging: supporting 4K resolution at 72FPS [22, 27] with sustained bitrates of 400–500Mbps [10, 29, 40] can quickly saturate wireless links [20]. Combined with stringent latency requirements, these bandwidth demands make VR streaming one of the most challenging and compelling applications.

To reduce wireless bandwidth, existing works exploit users' *spatial viewing patterns*, either by streaming only the current viewport or by allocating more pixels to visible regions through optimized projection, rendering, and foveated encoding [5, 9, 11, 13, 15, 29, 31, 40]. While these approaches can reduce bitrate consumption, they rely on accurate viewport or gaze predictions, which are highly scene-dependent. Even a small prediction error can introduce visual artifacts and severely degrade users' Quality of Experience (QoE). This raises a key question: *can we achieve satisfactory VR streaming without excessive bandwidth demands in a way that is universal across any VR scene?*

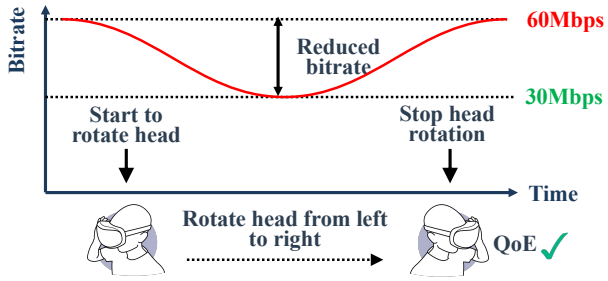


Figure 1: MARS **reduces the bitrate from 60Mbps to 30Mbps as the user's head moves from left to right, saving bandwidth while maintaining QoE; 30Mbps is sufficient to avoid perceptible visual differences.**

Adaptive bitrate (ABR) [19, 37] is widely used in Video-on-Demand (VoD) systems to dynamically adjust the bitrate to network conditions. Inspired by this, we leverage users' head motions to guide bitrate adaptation. During head movements, users' *dynamic visual acuity* decreases due to motion masking, making them less sensitive to frame quality [7, 11, 30]. This provides an opportunity to reduce bitrate without impacting perceived quality. As shown in Figure 1, when a user turns their head from left to right, the system can lower the encoding bitrate from 60 Mbps to 30 Mbps. If visual quality remains above the minimum threshold, the user's QoE is preserved while saving bandwidth. Leveraging this principle, we present MARS, a motion-aware adaptive bitrate system that saves bandwidth while preserving QoE. While the underlying idea of MARS is intuitive, its realization is non-trivial and comes with two challenges: precise synchronization of head motion with the trigger for bitrate adaptation, and selection of bitrate reductions that balance bandwidth savings with visual quality.

Precise Synchronization. Precise synchronization is required between head motion and the trigger of bitrate adaptation. When the head starts moving, the sender must begin reducing the bitrate, and when the motion stops, the reduction must cease. If either is mistimed, QoE may degrade and resources may be wasted due to incorrect bitrate selection. As illustrated in Figure 1, if bitrate reduction begins after the user completes a head movement, the adjustment leads to reduced quality frames and breaks QoE. Moreover, delays in delivering head motion updates can further misalign adaptation. For example, if bitrate reduction continues after the head has stopped due to delayed feedback, the user experiences unnecessary quality degradation. This raises a key challenge: *how can we precisely synchronize adaptation with head motion under network dynamics?*

Adaptive Bitrate and QoE Tradeoff. Once synchronization is achieved, the next question is how much the bitrate can be reduced. Excessive reduction can be perceptible and degrade QoE, requiring a careful trade-off between visual quality and bandwidth savings.

Importantly, bitrate reduction should be motion-aware: faster head movements allow larger reductions due to lower visual sensitivity, while slower motions require more conservative adjustments. This leads to a key challenge: *how can we dynamically adapt bitrate to balance visual quality and bandwidth efficiency?*

To address these two challenges, MARS employs two solutions: a kinematics-based synchronization mechanism and a perceptually guided bitrate adaptation algorithm.

Kinematics-Based Synchronization Mechanism. To achieve precise synchronization and speed-aware bitrate adaptation, we design a kinematics-based state machine. When a user's head starts moving, the motion exhibits acceleration, and as it comes to a stop, it decelerates. We leverage these two phases to trigger the bitrate adaptation. To determine the appropriate level of adaptation, the system uses the head motion speed: faster motions allow more aggressive bitrate reduction, while slower motions require more conservative adjustments. This results in a four-state synchronization mechanism that jointly captures motion phases and adaptation levels. Finally, we account for the delayed update, which experiences Motion-to-Photon (MTP) latency, i.e., the temporal interval between a user input's reception by the server and the visual presentation of the corresponding rendered frame on the VR headset. Since MTP can be measured and known at the sender, we can leverage MTP information to anticipate head motion transition and proactively reduce or raise the bitrate.

Perceptually-Guided Bitrate Adaptation. To achieve an appropriate QoE tradeoff, we design a perceptually guided bitrate adaptation algorithm that maps head motion speed to the minimum bitrate required to maintain target visual quality, based on the Just Noticeable Difference (JND) principle [7]. During head motion, the key challenge is determining how much bitrate reduction can be applied without perceptible quality loss. To this end, we leverage prior profiling that characterizes JND thresholds as a function of motion speed [11]. Faster head motions tolerate larger quality degradation, while slower motions require higher fidelity. To translate perceptual thresholds into bitrate decisions, we perform offline profiling that maps bitrate levels to corresponding quality metrics (e.g., PSPNR [11]). Given a specific motion speed, we select the lowest bitrate whose quality remains within the acceptable JND bound. For example, if the current bitrate is 10 Mbps and the JND threshold allows a certain degradation due to motion, we can derive a lower bitrate whose corresponding quality degradation is imperceptible.

Taken together, MARS is designed as a lightweight, plug-and-play component that integrates seamlessly into existing VR streaming systems without adding system overhead. To evaluate its effectiveness, we implemented an end-to-end streaming prototype of MARS on top of ALVR [6]. We conduct a comprehensive user study to evaluate MARS, finding that 21 out of 31 participants perceived it as visually indistinguishable from a fixed-bitrate baseline. In addition, we developed an end-to-end emulation pipeline using the NVIDIA Codec SDK [26] to benchmark MARS against state-of-the-art approaches. Experimental results show that MARS reduces bandwidth consumption by 10–19% compared to the fixed-bitrate baseline. In summary, we make the following key contributions:

- To address the bandwidth-intensive requirements of VR streaming that can degrade QoE, we propose a practical adaptive bitrate algorithm that utilizes the head motion as a universal solution to a plethora of VR applications.
- We tackle two key challenges: precise synchronization to avoid QoE degradation using a kinematics-based state machine, and selecting the optimal bitrate to achieve the best QoE–bandwidth trade-off via a perceptually guided approach.
- We design and implement MARS, an end-to-end bandwidth-efficient VR streaming system, and demonstrate through user studies and benchmarking that it reduces bandwidth consumption without compromising users' QoE.

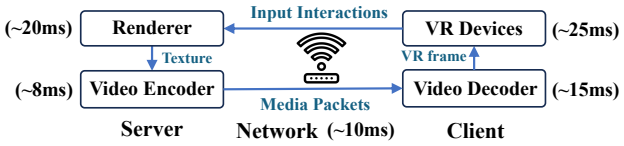


Figure 2: VR streaming workflow and latency breakdown.

2 Background and Motivation

In this section, we will present the overview of wireless VR streaming, existing approaches to reduce the bandwidth consumption in VR streaming, and motivate the design of MARS.

2.1 Untethered VR Streaming

Recent standalone headsets, such as Meta Quest [22] and PICO [27], support wireless VR streaming, where rendered scenes are delivered from a powerful workstation to the headset. By cutting the physical tether, these systems provide users with greater mobility than traditional tethered VR systems. A typical untethered VR streaming system, illustrated in Fig. 2, consists of two main components: a server that runs the VR application and a VR headset that serves as both the input and output interface. On the input side, the headset leverages onboard sensors, such as gyroscopes and accelerometers, together with Bluetooth controllers [14, 21, 22], to capture user actions (e.g., head orientations and hand gestures). These inputs are transmitted to the server, where they are processed by the VR application pipeline to generate updated scene frames. The rendered frames are then encoded and streamed back to the headset for display. Several practical systems have been developed to support wireless VR streaming, including ALVR [6] and Steam Link [2]. In VR applications with physics-driven simulation, the virtual environment evolves continuously even in the absence of explicit user input. As a result, the server must render frames in real time regardless of user actions.

MTP latency. MTP latency is a key performance metric in VR systems and is defined as the elapsed time between when a user input is received by the server and when the corresponding rendered frame is streamed back and displayed to the user. To better understand MTP latency in VR streaming, we deployed a typical VR streaming system where a commercial GPU (4080RTX) workstation and a VR headset (PICO3 [27]) are co-located under a WLAN. We use ALVR [6], the open-sourced streaming framework to transmit the rendered VR frames to the VR headset. Our measurements show that MTP latency consists of several components: server-side rendering latency (~20 ms), encoding latency (~8 ms), network latency (~10 ms), and client-side processing latency (~40 ms), including

decoding, queuing, and synchronization delays [6, 31]. As a result, the end-to-end MTP latency is at least 70 ms.

2.2 Existing Bandwidth Reduction Approaches

Streaming VR frames is highly bandwidth-intensive, and maintaining an immersive experience typically has higher display requirements than conventional video streaming, e.g., around 72 fps and resolutions up to 4K [22, 27]. Reducing bandwidth in VR streaming has been actively investigated in previous work [5, 6, 9, 13, 15, 29, 31, 36, 39, 40], which can be divided into two categories.

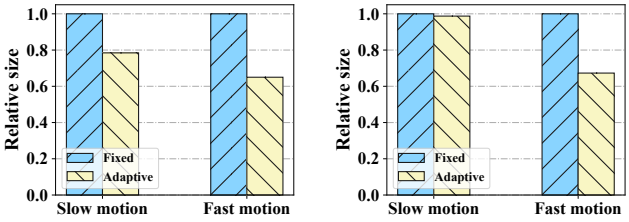
Viewport-aware streaming. This approach is motivated by the observation that a viewer sees only a small part of the full 360-degree scene at any given moment. Based on this insight, systems such as ALVR [6] stream only the user's current viewport (100° × 100° in degrees) rather than the entire scene to reduce the bandwidth consumption. However, this design imposes stringent latency requirements where the streamed content must arrive before the viewer moves outside the current viewport [31]. This requirement becomes particularly challenging when the MTP latency is high. To mitigate this issue, prior work [29, 36, 39, 40] has explored *viewport prediction*, which estimates future viewing directions and enables the client to prefetch the corresponding viewports in advance. These prediction methods typically exploit head motion trajectories [29, 40] and visual information such as content saliency [39]. However, such techniques are effective for video-on-demand systems [29], where content can be preprocessed offline to facilitate prediction and prefetching [29, 39]. Viewport prediction is less feasible for real-time VR streaming. *In real-time VR, the environment evolves dynamically in response to user interactions and application state at a very short horizon. A small error in viewport prediction will lead to visual artifacts and significantly damage users' experience.*

Foveated compression. Foveated compression is another major approach to reducing bandwidth in VR streaming. It is motivated by a fundamental property of human vision that although a human's visual field spans a wide range (220° × 135°), only a very small foveal region (<2°) [17, 28] can perceive fine detail with high acuity. Visual sensitivity declines significantly in the periphery. By exploiting this property together with real-time eye tracking, foveated systems reduce bandwidth by allocating resources unevenly across the viewport, preserving high quality in the user's gaze region while degrading peripheral regions progressively. Existing techniques mainly operate at two stages of the pipeline: foveated rendering [15], which adjusts rendering quality according to gaze, and foveated video encoding [5, 13], which allocates more bits to the foveal region and fewer bits to the periphery during compression. *This approach is complementary to ours and can be integrated with our design to further reduce bandwidth consumption.*

2.3 Motivations and Challenges

In this part, we demonstrate the potential of exploiting the spatial viewing characteristics to reduce bandwidth cost and highlight the challenges of applying it to real-time VR streaming.

Motion-aware streaming. The above approaches primarily exploit the *spatial characteristics* of human perception on VR contents. In contrast, during rapid head motion, users' dynamic visual acuity is generally lower than that under static viewing conditions [7, 11, 30]. As a result, users can often tolerate lower visual detail during movement. Motivated by this observation, we can adapt the encoding bitrate of the rendered frame according to the user's head movement speed to reduce bandwidth consumption while preserving QoE. To see this, we conducted a preliminary user study. Specifically, we selected two 8K VR videos from YouTube [24], namely, *Dragon* and *Mycraft*, and used five viewing trajectories collected from an open-sourced dataset [16]. By integrating the viewing traces with the selected videos, we generated 10 VR video clips in total, where each scene change follows the corresponding viewing trajectory. Each clip lasts 15 seconds. The original downloaded videos have a bitrate of approximately 100 Mbps. We then applied a simple rate adaptation policy (outlined in Figure 3c) [11] to re-encode each frame of the video based on the head movement speed. To evaluate the perceptual impact, we presented participants with a combined video where the adapted clip was concatenated with a fixed-bitrate baseline encoded at 60 Mbps side-by-side in a VR headset.[1] According to participant feedback, no perceptible difference in visual quality was observed between the fixed and adapted versions. Fig. 3 further reports the normalized sizes of the re-encoded *Minecraft* and *Dragon* videos under two representative viewing trajectories with slow (22 degrees/s) and fast (54 degrees/s) head movement, respectively. As shown, even a simple rate adaptation policy [11] reduces bandwidth consumption by up to 16.8% without degrading user experience. The encoded data size of *Dragon* is larger than *Minecraft* as it involves more complex and dynamic scenes.



(a) VR App. 1: Minecraft. (b) VR App. 2: Dragon.

(c) Bitrate adaptation policy.

Speed (degrees/s)	0–10	10–20	20–40	40–80	>80
Bitrate (Mbps)	60	50	40	30	20

Figure 3: Reducing video size by 16.8% does not affect viewers' perceptual quality.

VoD *vs.* Real-time VR. This principle has previously been explored in 360-degree VoD systems (e.g., Pano [11]) where a 360 video is partitioned into tile chunks, and each tile chunk is further pre-encoded into multiple quality levels. The tile chunks that lie within the users' predicted viewport are transmitted at a selected bitrate determined by head motion speed and available bandwidth. However, a fundamental temporal mismatch exists between bitrate selection and head motion as the bitrate decisions are made at the chunk level (in seconds), whereas head motion updates occur at the frame level (in milliseconds). Therefore, a single bitrate assigned to

[1] More detailed user study setup and evaluation results are described in §4.1.

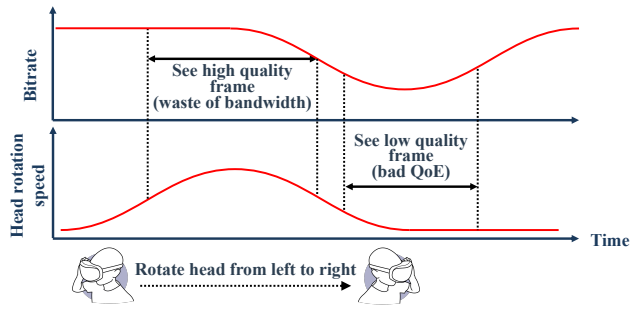


Figure 4: Misalignment between bitrate adaptation and head motion leads to *poor experience* and *bandwidth waste*.

a video chunk cannot adequately account for the full range of head motion that occurs within that chunk's duration.

Challenges. In real-time VR streaming, bitrate adaptation can be performed at a much finer granularity. However, it requires more accurate alignment between the bitrate adaptation and the instantaneous head motion. This becomes much more challenging in the presence of MTP latency, which can introduce two key issues. First, it can lead to *bandwidth waste*. As illustrated in Fig. 4, the viewer rotates their head from left to right and then stops. Initially, the viewer moves their head slowly, so the system selects a relatively high bitrate for the upcoming VR frame. However, this request must go through the end-to-end pipeline where the server receives the request, renders and encodes the frame at high quality, and then sends it back to the client. By the time the frame is displayed, the user's head motion may already have transitioned to a fast state (i.e., high head rotation speed), during which such high visual quality is no longer necessary. As a result, the system spends bandwidth on streaming unnecessary high quality frame, as a more compressed frame would have been perceptually acceptable. As time goes by, when the viewer transitions from fast motion to slow motion, the user may have a bad viewing experience. This happens because when the viewer slows down his/her head movement, he/she still see low-quality frames that were previously requested during the fast motion. In this case, the viewer must wait a longer time until newly rendered high-quality frames arrive. Under MTP latency, a frame requested at time t is not displayed until $t + \Delta$ which means that the bitrate decision must anticipate the user's motion state at display time, not at request time. ***This motivates the core challenge we aim to address in this paper: when and how to perform bitrate adaptation that captures viewers' head motion dynamics in the presence of MTP latency?***

3 MARS Design

3.1 Overview

Fig. 5 illustrates the high-level architecture of MARS, which consists of a wireless VR client and a rendering server. The client periodically reports head pose to the server. Based on this feedback, the server renders, encodes, and streams the corresponding video frame back to the client, which then decodes and displays the received frame on the headset. In the figure, the light blue and yellow blocks denote the conventional VR streaming pipeline, while the purple blocks highlight the key components introduced by MARS. MARS

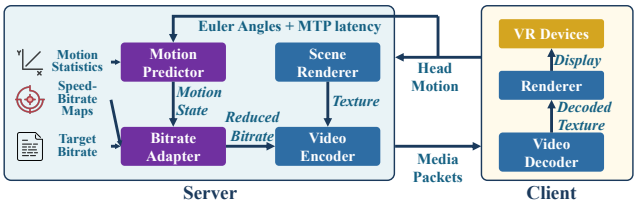


Figure 5: System overview.

augments the server with two motion-related modules. First, the *motion predictor* leverages the designed *kinematics-based state machine model* to capture the viewers' motion dynamics for accurate and timely bitrate adaptation. Then, the *bitrate adapter*, which employs a *perceptually-guided rate adaptation algorithm*, determines an appropriate reduced bitrate based on the anticipated motion dynamics and the target bitrate specified by the application. The design and functionality of these components are presented in the following sections.

3.2 Kinematics-Based Synchronization

In this section, we describe how we proactively predict users' motion trends to mitigate the mismatch between the bitrate adaptation and head motion induced by MTP latency (§2.3). To this end, we present a kinematics-based state machine model that captures not only the current motion intensity but also its future trajectory, enabling timely bitrate adaptation ahead of motion changes. Specifically, in Fig. 6, we categorize users' head movement into four states.

- *Slow* is a stable state where viewers exhibit slow or negligible head movement. Users typically remain in this state for the majority of the time (as shown in Table 1), requiring a high bitrate to ensure optimal visual quality.
- *Fast* is a dynamic state characterized by rapid head movements sustained for a short duration. During this state, the bitrate can be safely reduced as viewers are less sensitive to quality degradation.
- *Speed-up* is a transitional state in which the viewer's head motion accelerates from *Slow* state toward *Fast* state.
- *Decelerate* is a transitional state in which the viewer's head motion slows down, moving from *Fast* state back to *Slow* state.

The transitions among these states follow natural human head movement dynamics. We initially characterize each state transition using two metrics: viewport speed and viewport acceleration. Intuitively, speed captures the current motion intensity, while acceleration indicates the motion trend. Starting from *Slow* state, the motion enters *Speed-up* state when the acceleration exceeds a_{up} and the viewport speed rises above v_{slow}, indicating that the motion is starting to build up. From *Speed-up* state, the motion either transitions to *Fast* state if the speed further exceeds v_{fast}, or switches to *Decelerate* state if the acceleration becomes negative. Similarly, *Fast* state transitions to *Decelerate* state when the acceleration and speed fall below a_{down} and v_{fast}, respectively, and *Decelerate* state returns to either *Slow* or *Speed-up* states, depending on whether the motion continues to slow down or starts increasing again. We will discuss the parameter settings in §4.1.

To further ground the model in real viewing behavior, we perform offline profiling using the public dataset from [16], which contains head orientation traces from 3,791 viewing sessions across

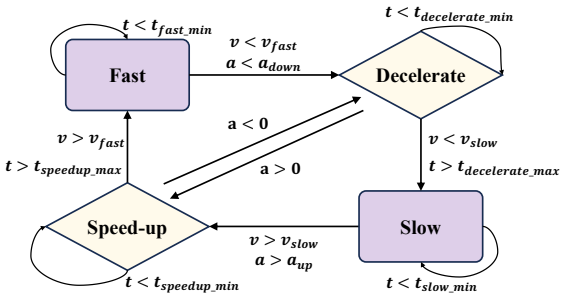


Figure 6: Illustration of head motion state transitions.

Table 1: Statistics of dwelling time of motion states.

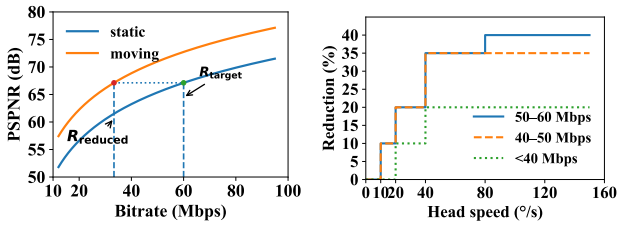
Dwelling Time	# Samples	Min (ms)	Max (ms)	Avg (ms)
Fast	62450	67.0	1598.8	489
Slow	95775	74.1	11459.5	1576.8
Decelerate	136376	25.5	773.2	231.3
Speed-up	138560	32.5	708.0	208.9

88 360-degree videos. We first compute viewport speed at a 10ms granularity and smooth it using an exponential moving average (EMA). We then derive acceleration from the smoothed speed and apply EMA again to avoid noisy data points. Using the resulting speed and acceleration traces, we label the motion state at each timestamp and analyze the dwelling time of each state, as summarized in Table 1. This table shows that viewers remain in *Slow* state for most of the time as expected, while the other three states are comparatively short-lived. We further augment the state-transition model with the constraints of dwelling time. Specifically, each state is assigned a minimum dwelling time t_{min}, during which transitions are suppressed to avoid oscillation caused by noisy speed or acceleration measurements. For *Fast*, *Speed-up*, and *Decelerate* states, we additionally impose a maximum dwelling time to ensure timely exit from transient states.

During runtime, we compute the instantaneous viewport speed and acceleration from the pose information reported by the client while tracking the time spent in the current motion state. If the head motion enters into either *Speed-up* or *Decelerate* state, bitrate adaptation will be triggered, and the anticipated viewport speed is calculated as $v_{anti} = v_{curr} + a_{curr} * MTP$, which is used for the bitrate selection (will be discussed later). In this case, MARS can anticipate users' future motion and proactively adapt the bitrate ahead to address the synchronization issue described in §2.3. For example, when the system detects the head starts to move, it knows that after the MTP delay, the viewer will enter a Fast state. Therefore, it can trigger the bitrate reduction ahead of time. Furthermore, by combining with dwelling time constraints, the state model further suppresses oscillation caused by transient signal noise, which ensures temporally stable and reasonable bitrate decisions throughout the VR session.

3.3 Perceptually-Guided Rate Adaptation

In this section, we describe how MARS selects an appropriate bitrate given the predicted motion state. The original given bitrate depends on dynamic network conditions and VR application requirements. The selected bitrate (reduced bitrate with respect to the given bitrate) should preserve the user's perceived visual quality while reducing bandwidth consumption as much as possible.



(a) Perceptual quality *vs.* bitrate. (b) Reduced bitrate *vs.* speed.

Figure 7: Mapping viewport speed to bitrate reduction.

We determine the reduced bitrate $R_{reduced}$ as follows. Let $Q_{base}(r)$ denote the perceptual quality at the bitrate r when the user motion is static. For a motion speed v and a target bitrate R_{target}, $R_{reduced}$ is determined as:

$$R_{reduced} = Q_v^{-1}(Q_{base}(R_{target})), \quad (1)$$

where $Q_v(r)$ is the perceptual quality at bitrate r when viewport moving speed is v. Here, the quality function $Q(\cdot)$ can be instantiated using a metric such as Peak Signal-to-Perceptible-Noise Ratio (PSPNR) [11], which serves as one perceptual metric that inherently discounts visually imperceptible distortions in the presence of the head motion. Grounded in the JND principle, PSPNR quantifies how much bitrate can be reduced at a given head rotation speed without degrading users' perceptual quality. As illustrated in Fig. 7a, $R_{reduced}$ represents the smallest bitrate at speed v that achieves the same perceptual quality as R_{target} in the static case. Since this classic perceptual metric is typically concave with respect to bitrate, our approach leads to larger absolute bitrate reductions when R_{target} is high (e.g., network condition is good) and smaller reductions when R_{target} is low.

We perform offline profiling to construct the relationship between the bitrate and PSPNR, i.e., $Q(\cdot)$. Specifically, we collect 17 8K VR videos from YouTube and encode each video at multiple bitrate levels, i.e., {10, 15, ..., 55, 60} Mbps. For each encoded version, we apply five viewing trajectories from [16] and compute the averaged PSPNR [11]. We then group viewport speeds into bins of [0,10), [10,20), ..., [80,100), and ≥ 100 degrees/s, and derive a PSPNR–bitrate curve for each bin. These profiled curves characterize the maximum reduced bitrate allowed for a given target bitrate without causing perceptual quality degradation under different motion speeds, as illustrated in Fig. 7b. During runtime, we select the bitrate based on the predicted transition state and corresponding predicted viewport speed according to Fig. 7b.

4 Experimental Results

4.1 Subjective Evaluation

In this section, we evaluate whether MARS can achieve perceptual quality comparable to baseline schemes that use a fixed bitrate.

Testbed. To evaluate MARS, we build a testbed in a WLAN environment. We prototype MARS on top of the open-source VR streaming platform ALVR [6], extending its *TrackingManager*, *StatisticsManager*, and *BitrateManager* to extract head-related dynamics, obtain MTP latency as the prediction horizon, and apply the proposed bitrate adaptation, respectively. The server runs on a laptop with

Table 2: Parameter Setup

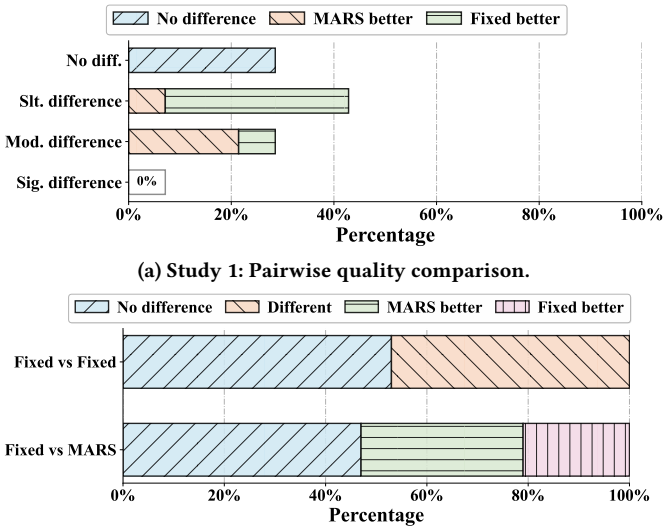
Notation	Description	Values
v_{slow}	Minimum viewport speed where head movement is considered dynamic	20°/s
v_{fast}	Minimum viewport speed at which head movement is classified as fast	60°/s
a_{up}	Acceleration threshold for transitioning from *Slow* to *Speed-up*	+10°/s^2
a_{down}	Acceleration threshold for transitioning from *Fast* to *Decelerate*	-10°/s^2
$t_{decel,max}$	maximum dwelling time for Decelerate	773.2
$t_{speedup,max}$	maximum dwelling time for Speed-up	708.0
$t_{fast,min}$	minimal dwelling time for Fast	67.0
$t_{slow,min}$	minimal dwelling time for Slow	74.1
$t_{decel,min}$	minimal dwelling time for Decelerate	25.5
$t_{speedup,min}$	minimal dwelling time for Speed-up	32.5

an NVIDIA RTX 4080 GPU, an Intel i7 CPU, and 32 GB of memory. The client is a PICO NEO3 VR headset [27] connected via a Wi-Fi 802.11ax access point.

Parameter setup. The target frame rate, resolution, and bitrate are set to 72 FPS, 4K, and 60Mbps. We use HEVC for frame encoding. We adopt CBR (constant bitrate) mode to produce more stable bitrate streams and reduce the risk of playback stalls during streaming. The parameters for state transitions are summarized in Table 2. In particular, the dwelling time for each state is determined based on profiled statistical information, as shown in Table 1.

User study setup and results. We conducted two formal user studies with a total of 31 participants.[2] The self-reported age range was 20–34 years, comprising 4 females and 27 males. Prior to each study, all participants familiarized themselves with the VR device and the ALVR streaming system using a virtual tourism application for 5 minutes. During these two studies, participants engaged with two VR applications of varying scene complexity. Application 1 is *Secret Shop* [35], which features abundant lighting, color, and texture resources. Application 2 is the UE-packaged test resource *VR Template* [34], with simpler lighting and color schemes, yet containing rich line edges and interactive objects.

- The first study involved 14 participants where each person was assigned two VR streaming schemes in randomized order: MARS and *Fixed*. In *Fixed*, the target bitrate remains constant during playback. Participants played each scheme for 5 minutes and were unaware of which scheme they were using. After completing both sessions with a short break, participants rated whether one scheme provided better visual quality with various scales: *no difference, slight difference, moderate difference, or significant difference*. The result is shown in Fig. 8a. Out of the 14 participants, 8 reported either no noticeable quality difference or MARS provided better visual quality compared to *Fixed*. However, 6 participants rated MARS as not performing better than the baseline. Notably, no participant reported a significant difference between the two schemes. Based on our follow-up discussions with participants, random network fluctuations and occasional stall events contributed to lower ratings in some cases. These results suggest that MARS performs no worse than *Fixed* in terms of visual quality. However, the pairwise comparison design cannot isolate whether the reported differences reflect genuine perceptual quality gaps or a response bias, where participants tend to favor one option over the other in the absence of perceivable difference.
- To eliminate this bias, we conducted a second user study with 17 participants, each performing one experimental round for each of the two applications. In each round, participants were presented with one of two scheme combinations MARS vs. *Fixed* or *Fixed* vs. *Fixed*. without being told. Each participant completed 2 sessions per scheme combination, with each session lasting 1~2 minutes. Rather than scoring the difference at scale, participants gave only a binary response, i.e., whether they perceived a quality difference between the two schemes. The result is shown in Fig. 8b. We first examine the baseline case (first row), where participants were presented with two identical *Fixed* schemes. Despite the two schemes being exactly the same, 47% of participants still reported perceiving a quality difference. This result shows that nearly half of all responses in a pairwise comparison (Fig. 8a) are attributable to response bias rather than real perceptual difference. In the second row, we compare MARS and *Fixed* where 53% of participants reported there is a perceivable difference between the two schemes. Among the 53% who did report a difference, 32% judged MARS as the better scheme. Most importantly, the fraction of participants perceiving a difference barely increases from 47% (*Fixed vs. Fixed*) to 53% (*Fixed vs.* MARS). This marginal increase is well within the range of the established response bias, which validates that the bitrate updated by MARS is perceptually transparent to users.



(a) Study 1: Pairwise quality comparison.

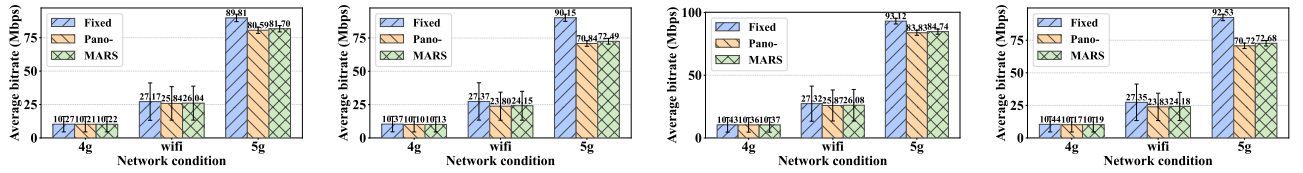
(b) Study 2: Bias-controlled quality comparison.

Figure 8: User study results. (a) Slt., Mod., Sig. stand for slight, moderate, and significant, respectively.

4.2 Emulation Results

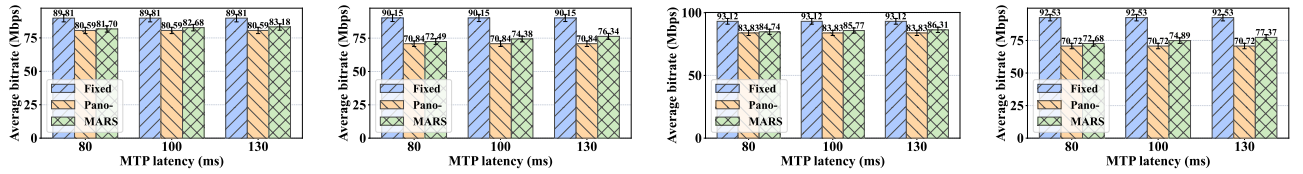
In this section, we use our emulation pipeline to benchmark existing schemes against MARS in terms of bandwidth reduction.

[2]Before engagement, all participants were provided an overview of the study and consented to participate per institutional ethics and privacy policies.



(a) Motion trace #1, App #1 (b) Motion trace #2, App #1 (c) Motion trace #1, App #2 (d) Motion trace #2, App #2

Figure 9: Bandwidth consumption *vs.* network conditions.



(a) Motion trace #1, App #1 (b) Motion trace #2, App #1 (c) Motion trace #1, App #2 (d) Motion trace #2, App #2

Figure 10: Bandwidth consumption *vs.* MTP latency.

Table 3: Network Traces.

Dataset	# of traces	Avg BW (Mbps)	Max BW (Mbps)
WiFi	96	27.58	86.61
4G	24	10.48	45.00
5G	30	93.04	150.00

Network trace. We collect network throughput traces from three datasets covering 4G [18], Wi-Fi [18], and 5G [25], comprising 24, 96, and 30 traces, respectively. In total, the dataset consists of 150 traces. Among them, the 4G traces exhibit the lowest average bandwidth. For the 5G dataset, we exclude traces whose maximum throughput exceeds 150 Mbps to avoid over-provisioning. Summary statistics are reported in Table 3.

Video and head movement dataset. We use a large head-movement dataset [3] containing 56 videos, and each video is viewed by around 50 participants. Head poses are recorded at 10 Hz in terms of yaw and pitch. Roll is omitted because viewers rarely rotate around that axis [31]. Since no public dataset provides high frame rate and high resolution VR videos, we self-collected 17 VR videos at 8K and 60 FPS using YouTube download tools [38]. These videos cover diverse content types, ranging from relatively static scenes such as the moon landing [1] to highly dynamic scenes.

Emulation setup. We build an emulation testbed on top of E3PO [12] and NVIDIA's Video Codec SDK (NvCodec) [26]. E3PO serves as a VR video streaming simulator. Given a raw 8K, 60 FPS panoramic video and a head-motion trace, E3PO extracts a 2K viewport-only video via resampling, rotation, and projection. This is then fed into our NvCodec-based encoding pipeline, where frames are processed sequentially at the playback rate (one frame per 16.7 ms). For each frame, the pipeline computes motion speed, determines the target encoding parameter via the current motion state and speed-to-bitrate mapping (shown in Fig. 7b), and applies it through NvCodec's *Reconfigure* API.

Baseline schemes. We consider two baselines: *Fixed* and *Pano-* [11]. *Fixed* is a fixed-bitrate policy that assigns the same target bitrate to every frame, regardless of viewport speed. For *Pano-*, we adopt the JND curve from [11] to construct a discrete speed-to-bitrate mapping, as shown in Figure 3c. To ensure a fair comparison and avoid additional system overhead, we disable Pano's content-dependent features, such as luminance and depth-of-field cues, and retain only its speed-based mapping. Specifically, *Fixed* determines the target bitrate solely based on the available bandwidth, while MARS and *Pano-* further reduce the bitrate based on the motion.

Impact of network bandwidth. Fig. 9 summarizes the bandwidth reduction achieved by MARS under different network conditions. Specifically, App 1 and App 2 are two representative VR applications, where App 2 contains more complex scenes and richer textures. Motion trace 1 and Motion trace 2 are two representative head-motion traces, where Motion trace 2 has a larger average viewport speed. Several insights can be drawn from the results. First, MARS achieves larger bandwidth savings under more favorable network conditions. For example, in Fig. 9a, it reduces bandwidth by 10% under 5G, whereas the gain under 4G is almost negligible. This is because when the available bandwidth is high, the VR frame is rendered at a higher bitrate, leaving more room for MARS to reduce bitrate without causing perceptible quality degradation. In contrast, under limited bandwidth, there is little space for further bandwidth reduction. Second, by comparing Fig. 9a and Fig. 9b, we observe that more intensive head motion enables up to 19% bandwidth savings, which confirms the effectiveness of incorporating viewport motion into bitrate adaptation. Third, by comparing Fig. 9a and Fig. 9c, we can find that content complexity also affects the achievable bandwidth reduction. Under 5G network, App 1 and App 2 achieve bandwidth reductions of 10% and 9%, respectively. The smaller gain for App 2 is due to its more complex and highly textured scenes, which require higher encoding precision to preserve perceptual quality. Finally, compared with *Pano-*, MARS achieves a comparable overall bitrate reduction, although *Pano-* performs slightly better. This difference arises because MARS adopts a more conservative adaptation policy that intentionally avoids overly aggressive bitrate reduction in order to mitigate the impact of MTP delay. In contrast, *Pano-* prioritizes instantaneous bandwidth savings, which may introduce temporal quality fluctuations during fast motion.

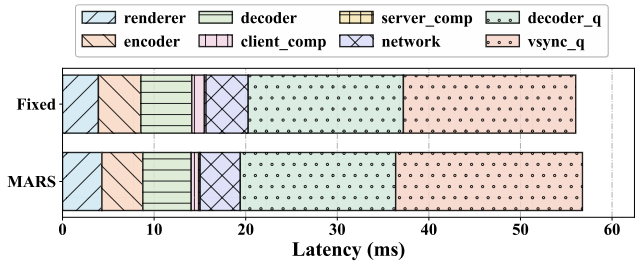


Figure 11: End-to-end latency breakdown.

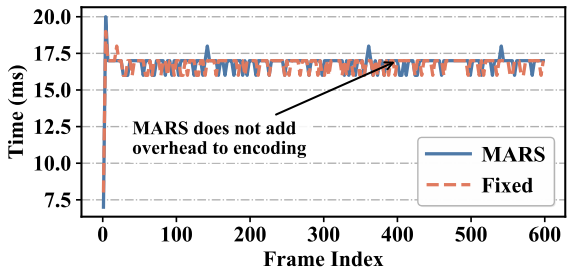


Figure 12: Encoding frame interval.

Impact of network latency. Fig. 10 plots bandwidth reduction versus MTP latency. We observe that as MTP latency increases, the bandwidth savings achieved by MARS over the *Fixed* baseline decrease. This trend arises from our latency-aware state-transition design: under larger MTP, bitrate adaptation must be more conservative. For example, the transition from *Fast* to *Decelerate* needs to be triggered earlier, so that the bitrate is raised in advance to compensate for the additional delay.

4.3 System overhead

Fig. 11 reports the MTP latency breakdown for MARS and *Fixed*. Interestingly, the dominant contributors are server-side rendering time, client-side queueing time, and synchronization time (denoted by *vsync_q*). The elevated synchronization delay is largely caused by a frame-cadence mismatch where the server renders the frame at 60 FPS in real-time but the headset refreshes at over 72 Hz, causing decoded frames to wait until the next display refresh. The queueing delay on the client side is also non-negligible, because frames typically accumulate in the playback buffer before presentation to avoid jitter. Because the experiments run on a local network, transmission latency is not a bottleneck.

To further verify the system overhead of MARS, we use NvCodec to evaluate the impact of adjusting the encoder rate parameters on the encoding latency of individual video frames. For the input image sequence encoded in constant bitrate mode at 60 fps, the target bitrate was switched between 45 Mbps and 60 Mbps every 30 frames. We recorded timestamps before and after each frame was submitted to the encoder, and computed the per-frame coding time accordingly, with the corresponding results illustrated in Fig. 12. It can be observed that MARS exhibits nearly identical single-frame coding latency relative to the *Fixed* baseline, which shows that the dynamic bitrate adaptation mechanism of MARS incurs no additional system overhead.

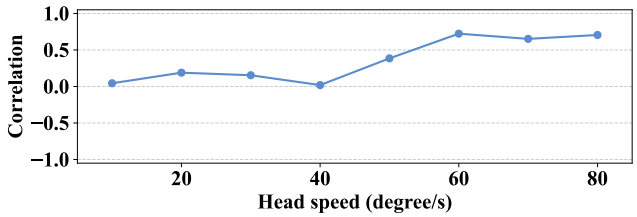


Figure 13: Impact of video content.

5 Discussion

In this section, we discuss the future optimization opportunities for MARS.

Integrate with video content. At this stage, MARS's motion-aware rate adaptation relies solely on pose signals, without explicitly modeling video content dynamics. To investigate whether content information should also be incorporated, we conducted a proof-of-concept experiment. Specifically, we extracted codec motion vectors (MVs) from the rendered video after injecting head-motion trajectories and measured their correlation with angular velocity using the Pearson correlation coefficient, as shown in Fig. 13. Interestingly, when viewport motion is slow (velocity $<$ 40 degrees/s), MV magnitude exhibits weak correlation with head speed, suggesting that scene dynamics may play a more important role in this regime. In contrast, when viewport motion is fast, MV magnitude is largely dominated by head movement. That is to say, viewport motion overwhelms intrinsic scene motion, making head speed alone a good proxy for bitrate adaptation. These results suggest a promising direction for future work where we can incorporate video-specific signals, such as motion vectors or optical flow, to further improve the rate adaptation algorithm.

Human personalization. MARS can be further enhanced through personalization. Demographic factors and prior VR experience, such as age, sex, and proficiency with VR, may correlate with viewing behaviors, including head-speed distributions, dwell times, and sensitivity to the quality degradation. Capturing these user-specific differences could provide personalized priors and further improve MARS's adaptation performance.

Deployment. MARS is a runtime-agnostic bitrate controller that can be packaged as a lightweight API or library and readily integrated into existing VR streaming stacks. Its integration requires only access to head-motion telemetry and the rate control interface, with no modifications to the game engine. Beyond ALVR [6], MARS can be readily extended to other VR streaming platforms such as Steam Link [2] and Meta's Air Link [23].

6 Conclusion

This paper presented MARS, a practical and deployable bitrate adaptation solution for real-time VR streaming. By leveraging users' temporal viewing patterns and incorporating a motion-aware bitrate adaptation algorithm, MARS mitigated the mismatch between the head motion and bitrate adaptation timing and struck a good balance between perceptual quality and bandwidth savings in the real-time VR streaming. We envision the idea of MARS can be readily applied to other applications like streaming camera assistant, where the temporal redundancy can be exploited to reduce the communication and computation cost.

References

[1] [n. d.]. Apollo 11 (8K 3D VR). YouTube. ([n. d.]). Available at: https://www.youtube.com/watch?v=Cq1MCT-zoZM. Accessed: 2026-03-13.

[2] Steam. 2015. Steam Link. https://store.steampowered.com/app/353380/Steam_Link/. (2015).

[3] 360VidStr . 2021. A-large-dataset-of-360-video-user-behaviour . https://github.com/360VidStr/A-large-dataset-of-360-video-user-behaviour. (2021).

[4] AIDAR Solutions. 2025. 8 Key Benefits of Virtual Reality Training in 2025. https://aidarsolutions.com/benefits-of-virtual-reality-training/. (2025).

[5] Ahmad Alhilal, Ze Wu, Yuk Hang Tsui, and Pan Hui. 2024. FovOptix: Human Vision-Compatible Video Encoding and Adaptive Streaming in VR Cloud Gaming. In *Proceedings of the 15th ACM Multimedia Systems Conference (MMSys '24)*. Association for Computing Machinery, New York, NY, USA, 67–77. https://doi.org/10.1145/3625468.3647612

[6] ALVR. Jul-2024. ALVR - Air Light VR. https://github.com/alvr-org/ALVR. (Jul-2024).

[7] Joseph L. Demer and Vicente Honrubia. 1995. Evaluation of vestibular and visual oculomotor function. *Otolaryngology–Head and Neck Surgery* 112, 1 (1995), 16–34. https://doi.org/10.1016/S0194-5998(95)70300-7

[8] Gabriel Zamora. 2024. The Best VR Games for 2024. https://www.pcmag.com/picks/the-best-vr-games. (2024).

[9] Ehab Ghabashneh, Chandan Bothra, Ramesh Govindan, Antonio Ortega, and Sanjay Rao. 2023. Dragonfly: Higher perceptual quality for continuous 360 video playback. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 516–532.

[10] Grand View Research. 2023. Virtual Reality (VR) Market Size And Share Report, 2030. https://www.grandviewresearch.com/industry-analysis/virtual-reality-vr-market. (2023).

[11] Yu Guan, Chengyuan Zheng, Xinggong Zhang, Zongming Guo, and Junchen Jiang. 2019. Pano: Optimizing 360 video streaming with a better understanding of quality perception. In *Proceedings of the ACM Special Interest Group on Data Communication*. 394–407.

[12] Yongqiang Gui, Yanyan Suo, Tian Zhang, Zhenhua Yu, and Shu Shi. 2023. Poster: E3PO-An Open Platform for 360 Video Streaming Simulation and Evaluation. In *Proceedings of the 2023 ACM on Internet Measurement Conference*. 716–717.

[13] Luke Hsiao, Brooke Krajancich, Philip Levis, Gordon Wetzstein, and Keith Winstein. 2022. Towards retina-quality VR video streaming: 15ms could save you 80% of your bandwidth. *ACM SIGCOMM Computer Communication Review* 52, 1 (2022), 10–19.

[14] HTC. Dec-2023. Compare VIVE Systems . https://www.vive.com/us/product/comparison/. (Dec-2023).

[15] Gazi Karam Illahi, Matti Siekkinen, Teemu Kämäräinen, and Antti Ylä-Jääski. 2021. Foveated streaming of real-time graphics. In *Proceedings of the 12th ACM Multimedia Systems Conference (MMSys '21)*. Association for Computing Machinery, New York, NY, USA, 214–226. https://doi.org/10.1145/3458305.3463383

[16] Yili Jin, Junhua Liu, Fangxin Wang, and Shuguang Cui. 2022. Where are you looking? A large-scale dataset of head and gaze behavior for 360-degree videos and a pilot study. In *Proceedings of the 30th ACM International Conference on Multimedia*. 1025–1034.

[17] E. Llapashtica et al. 2024. Effects of Post-Saccadic Oscillations on Visual Processing Times. *Proceedings of the Royal Society B* (2024).

[18] Gerui Lv, Qinghua Wu, Weiran Wang, Zhenyu Li, and Gaogang Xie. 2022. Lumos: towards Better Video Streaming QoE through Accurate Throughput Prediction. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. 650–659. https://doi.org/10.1109/INFOCOM48880.2022.9796948

[19] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with pensieve. In *Proceedings of the conference of the ACM special interest group on data communication*. 197–210.

[20] Ferran Maura, Francesc Wilhelmi, and Boris Bellalta. 2026. Evaluating Wi-Fi Performance for VR Streaming: A Study on Realistic HEVC Video Traffic. (2026). arXiv:cs.NI/2601.16950 https://arxiv.org/abs/2601.16950

[21] Meta. 2023. Expand your world with Meta Quest. https://www.meta.com/quest/. (2023).

[22] Meta. 2023. This is Meta Quest Pro . https://www.meta.com/quest/quest-pro/. (2023).

[23] Meta. 2024. Set up and connect Meta Quest Link and Air Link. https://www.meta.com/en-gb/help/quest/509273027107091/?srsltid=AfmBOooubeRMWwqUj_NLyQ2Rl4bHHBhgE4vLihNOAeTi2OJn7SvxgCaZ. (2024).

[24] META VR 360. 2022. 360° VR SQUID GAME - Red Light Green Light || 8K. https://www.youtube.com/watch?v=M3PCtL6rbUs. (2022).

[25] Arvind Narayanan, Eman Ramadan, Rishabh Mehta, Xinyue Hu, Qingxu Liu, Rostand AK Fezeu, Udhaya Kumar Dayalan, Saurabh Verma, Peiqi Ji, Tao Li, et al. 2020. Lumos5G: Mapping and predicting commercial mmWave 5G throughput. In *Proceedings of the ACM internet measurement conference*. 176–193.

[26] Nvidia. 2025. NVIDIA Video Codec SDK . https://developer.nvidia.com/video-codec-sdk. (2025).

[27] PICO. 2023. This is Pico Neo3 . https://www.picoxr.com/global/products/neo3-link. (2023).

[28] Jan M. Provis, Penelope L. Dubis, James Carroll, and Alfredo Dubra. 2013. Adaptation of the Central Retina for High Acuity Vision: Cones, the Fovea and the Avascular Zone. *Progress in Retinal and Eye Research* 35 (2013), 63–81.

[29] Feng Qian, Bo Han, Qingyang Xiao, and Vijay Gopalakrishnan. 2018. Flare: Practical Viewport-Adaptive 360-Degree Video Streaming for Mobile Devices. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom '18)*. Association for Computing Machinery, New York, NY, USA, 99–114. https://doi.org/10.1145/3241539.3241565

[30] John Ross, Maria Concetta Morrone, and David Burr. 2001. Changes in visual perception at the time of saccades. *Trends in Neurosciences* 24, 2 (2001), 113–121. https://doi.org/10.1016/S0166-2236(00)01685-4

[31] Shu Shi, Varun Gupta, and Rittwik Jana. 2019. Freedom: Fast Recovery Enhanced VR Delivery Over Mobile Networks. In *Proceedings of the 17th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '19)*. Association for Computing Machinery, New York, NY, USA, 130–141. https://doi.org/10.1145/3307334.3326087

[32] SHRM Labs. 2024. Why Companies Are Making VR/AR Job Training Central to 2025 Strategy. https://www.shrm.org/labs/resources/companies-making-vr-ar-job-training-central-to-strategy. (2024).

[33] Statista Market Insights. 2025. AR & VR - Worldwide. https://www.statista.com/outlook/amo/ar-vr/worldwide. (2025).

[34] Unreal Engine. 2026. VR Template. https://dev.epicgames.com/documentation/en-us/unreal-engine/vr-template-in-unreal-engine. (2026).

[35] Valve. 2023. Secret Shop (Dota 2) | The Lab VR. https://www.youtube.com/watch?v=pGlRlcLWXfI. (2023).

[36] Shibo Wang, Shusen Yang, Hailiang Li, Xiaodan Zhang, Chen Zhou, Chenren Xu, Feng Qian, Nanbin Wang, and Zongben Xu. 2022. SalientVR: Saliency-driven mobile 360-degree video streaming with gaze information. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. 542–555.

[37] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *Proceedings of the 2015 ACM conference on special interest group on data communication*. 325–338.

[38] yt-dlp . 2025. yt-dlp . https://github.com/yt-dlp/yt-dlp. (2025).

[39] Yinjie Zhang, Mingyuan Wu, Beitong Tian, Jiaxi Li, Bo Chen, Qian Zhou, and Klara Nahrstedt. 2023. SAVG360: Saliency-aware Viewport-guidance-enabled 360-video Streaming System. In *2023 IEEE International Symposium on Multimedia (ISM)*. IEEE, 36–43.

[40] Lingzhi Zhao, Ying Cui, Zhi Liu, Yunfei Zhang, and Sheng Yang. 2021. Adaptive Streaming of 360 Videos With Perfect, Imperfect, and Unknown FoV Viewing Probabilities in Wireless Networks. *IEEE Transactions on Image Processing* 30 (2021), 7744–7759. https://doi.org/10.1109/TIP.2021.3099741